



2026 年（第 19 届） 中国大学生计算机设计大赛

人工智能实践赛作品报告

作品编号： 2026041142

作品名称： 面向独居老人的隐私保护型异常行为识别与应急响应系统

填写日期： 2026/04/18

填写说明：

- 1、本文档适用于人工智能实践赛；
- 2、尽管预选赛仅完成部分工作，但是本文档需要针对决赛做出方案设计；
- 3、正文、标题格式已经在本文中设定，请勿修改；标题#的快捷键为“Ctrl+#”，正文快捷键为“Ctrl+0”；
- 4、本文档应结构清晰，突出重点，适当配合图表，描述准确，不易冗长拖沓；
- 5、提交文档时，以 PDF 格式提交；
- 6、本文档内容是正式参赛内容的组成部分，务必真实填写。如不属实，将导致奖项等级降低甚至终止本作品参加比赛。

目 录

第 1 章 作品概述.....	1
第 2 章 问题分析.....	1
2.1 问题来源.....	1
2.2 现有解决方案.....	2
2.3 本作品要解决的痛点问题	2
2.4 解决问题的思路.....	3
第 3 章 技术方案.....	4
3.1 隐私计算总体方案.....	4
3.2 面向 TEE 迁移的 SecureCore 模块边界设计.....	4
3.3 原型级完整性校验与远程证明迁移预留	5
3.4 事件摘要设计.....	6
3.5 差分隐私统计发布机制	7
3.6 系统整体架构与数据流	8
3.7 行为识别、追踪与风险分级	9
3.7.1 人体姿态检测.....	9
3.7.2 隐私保护机制.....	9
3.7.3 多目标追踪.....	10
3.7.4 风险分级引擎.....	10
3.7.5 通信协议与数据流.....	11
3.8 设计重点与难点	12
3.9 真实性说明	13
第 4 章 系统实现.....	13
4.1 系统整体架构.....	13
4.2 数据来源与数据集准备	14
4.3 模型训练与迭代优化	15
4.4 后端软件设计与模块实现	16
4.5 检测核心模块实现	16
4.6 前端软件设计与用户界面	17
4.7 系统部署方法.....	21
4.8 开发中遇到的困难与解决方法	22
第 5 章 测试分析.....	23
5.1 测试目标与测试环境.....	24
5.2 异常行为识别效果测试	25
5.2.1 跌倒检测效果.....	25
5.2.2 长时间静止检测.....	25
5.2.3 夜间异常检测.....	26
5.3 风险分级与规则引擎验证	26

5.4 SecureCore 逻辑隔离与完整性校验测试.....	27
5.4.1 架构边界验证.....	27
5.4.2 完整性校验与模型保护.....	27
5.4.3 事件摘要最小暴露.....	28
5.5 差分隐私统计发布测试.....	29
5.5.1 统计接口 DP 接入测试.....	29
5.5.2 DP 对系统可用性影响.....	30
5.6 系统实时性与稳定性测试.....	30
5.7 测试结论.....	31
第 6 章 作品总结.....	31
6.1 特色与创新点.....	31
6.2 推广应用场景.....	32
6.3 未来发展展望.....	32
参考文献.....	32

第1章 作品概述

随着我国人口老龄化进程加速，独居老人数量持续增长，其居家安全问题日益凸显。据世界卫生组织统计，65 岁以上老年人年跌倒发生率达 28%~35%，跌倒后未及时救助可能导致严重后果，如骨折、长期卧床、甚至死亡。传统解决方案如穿戴设备依赖用户主动佩戴、充电维护、脱下就失效，存在侵犯隐私、误报率高、缺乏智能预警等局限。

本项目构建了一套面向独居老人的隐私保护型异常行为识别与应急响应系统，核心功能包括：

- （1）异常行为识别——检测跌倒、长时间静止两类核心事件，并支持结合夜间时段等上下文条件进行场景化风险判断；
- （2）风险评估分级——风险等级（NORMAL/LOW/MEDIUM/HIGH）匹配差异化响应策略；
- （3）应急响应机制——三级响应框架，从本地提醒到紧急通知；
- （4）可视化监控——事件看板、统计分析、实时监测界面。

系统突出三大特色：隐私优先——本地处理所有数据，人脸自动模糊，遵循角色分级隔离，对平台仅传输事件摘要，对授权看护者侧开放经人脸模糊处理的实时验证视图；规则与机器学习融合——YOLO26 深度学习检测与规则引擎分级相结合；完整系统化——从采集到响应的全链路实现，具备实际部署能力。

第2章 问题分析

2.1 问题来源

随着老龄化加深，独居老人居家安全问题日益突出。老人一旦发生跌倒、昏厥或夜间异常活动，若不能被及时发现，可能导致较大风险。现有方案要么依赖穿戴式设备，存在忘戴、拒戴的问题；要么依赖持续视频监控，存在较强的隐私顾虑。

本项目拟解决的问题不是"替代医护"，而是构建一套低成本、轻部署、重隐私、可解释的家庭安全辅助系统。其研究意义主要体现在三个方面：

- (1) 针对独居老人常见风险事件，提供一个面向真实家庭场景的安全辅助方案；
- (2) 以隐私保护为核心约束，探索"本地识别+事件摘要上传"的轻量化实现路径；
- (3) 在无需专用硬件的条件下，形成一套完整的软件系统与算法闭环。

2.2 现有解决方案

目前市场上的老人监护产品可分为以下几类：

- (1) 穿戴式设备：如智能手环、跌倒报警器等，需老人主动佩戴。存在问题：老人容易忘戴或拒戴，设备故障时监测失效，且部分老人对佩戴设备存在抵触心理。
- (2) 传统视频监控：持续记录画面，需人工巡查或事后回放。存在问题：侵犯隐私，老人及家属接受度低，缺乏实时智能分析能力。
- (3) 智能监测系统：部分产品已应用计算机视觉技术，但存在以下不足：多数依赖持续上传原始视频，隐私风险大；仅实现单一跌倒检测，缺乏长时间静止、夜间异常等综合场景覆盖；告警机制单一，缺乏风险分级；系统完整性不足，难以直接演示部署。

现有智能看护产品普遍存在以下局限：多数依赖持续上传视频至云端，隐私风险较高；功能单一，仅支持跌倒检测；告警机制简单，缺乏风险分级响应。本系统在隐私保护、场景覆盖、风险分级等方面进行了针对性改进。

2.3 本作品要解决的痛点问题

基于上述分析，本系统聚焦解决以下痛点问题：

- (1) 穿戴式设备依赖问题：现有穿戴式设备需老人主动佩戴，存在忘戴、拒戴的问题，且设备故障时监测完全失效。本系统采用非接触式摄像头方案，无需老人配合佩戴。
- (2) 视频监控隐私顾虑问题：传统视频监控方案存在原始画面持续暴露的风险，老人及家属接受度低。本系统采用“本地处理优先、人脸模糊脱敏、对外仅传输事件摘要与匿名特征”的隐私分级策略，有效降低隐私侵扰风险。

(3) 检测场景覆盖不足问题：现有智能监测产品多聚焦单一跌倒检测，缺乏对长时间静止等高风险状态的覆盖，也未结合时间上下文进行场景化判断。本系统除跌倒检测外，实

现了基于时间感知的静止状态监测，能够结合夜间时段等上下文信息识别异常静止，提供更贴合实际看护需求的场景化判断。

(4) 告警机制单一问题：现有产品告警机制简单，缺乏风险分级与差异化响应。本系统实现低/中/高风险分级，匹配本地提醒、管理端告警、紧急通知的差异化响应策略，实际部署时可快速对接短信与电话等服务。

2.4 解决问题的思路

本系统采用“规则+机器学习”融合的技术路线，总体流程为：视频输入→本地匿名处理→特征提取→异常识别→风险融合→告警响应→日志展示。

检测目标：聚焦两类核心事件——跌倒识别（突然倒地、缓慢倒地后长时间不起）与静止状态监测（异常静止状态，并支持结合夜间时段等上下文信息进行场景化判断）。

隐私保护设计：采用“本地处理优先、人脸模糊脱敏、对外仅传输事件摘要与匿名特征”的隐私分级策略。通过 MediaPipe 人脸检测定位人脸区域，使用高斯模糊（51x51 核）对检测到的人脸进行匿名化处理，模糊区域按 50%比例扩展以覆盖周边特征，确保视频帧中的人物身份信息不可识别。

数据来源：采用 LE2I 公开跌倒检测数据集作为主要数据源，该数据集包含多视角跌倒视频，可用于验证模型泛化能力。同时自建少量模拟场景视频补充长时间静止、夜间异常等场景，自采数据严格遵循匿名化处理原则。

系统架构：采用 B/S 架构的四层安全设计——感知层（摄像头视频流接入）、安全处理层（逻辑隔离安全边界内完成检测、追踪、人脸匿名化与风险判定）、权威监督层（管理人员实时告警与脱敏事件查看）、社区网络化服务平台（差分隐私加噪后的聚合统计与趋势分析）。

第3章 技术方案

3.1 隐私计算总体方案

本作品采用了“安全逻辑隔离 + 差分隐私发布保护”的双层隐私增强方案。这两个部分的作用并不一样。安全逻辑隔离主要面向视频处理链路。它重点保护解码后的原始图像、人脸区域、人体框、轨迹、位移信息，以及风险判定之前的中间特征。差分隐私主要面向统计发布链路。它只对对外展示的聚合统计结果进行加噪处理。简单来说，前者解决的是“敏感视频数据怎样在受控边界内完成处理”的问题，后者解决的是“统计结果在发布时怎样降低个体信息泄露风险”的问题。

考虑到家庭和社区养老场景中的部署设备大多是通用计算平台，而不是专用的机密计算硬件，本系统在当前原型阶段采用了面向 TEE 迁移的 SecureCore 安全逻辑沙箱设计。具体来说，系统把核心敏感处理链路封装成独立的受控模块。这样一来，软件结构上就形成了比较清晰的隔离边界。与此同时，系统还通过最小接口暴露、完整性校验和事件摘要化输出等方式，对业务层面的隐私风险进行控制。

需要说明的是，当前原型还没有运行在真实的硬件 TEE 环境中。所以，本作品不宣称已经具备硬件级内存加密、硬件根信任或标准远程证明能力。本作品的创新点主要在于，系统已经构建了一个可以平滑迁移到未来机密计算环境中的模块化边界和数据流设计。

3.2 面向 TEE 迁移的 SecureCore 模块边界设计

SecureCore 遵循最小必要原则。它只接管数据敏感性最高的处理环节，如表 3-1 所示。根据本系统的数据敏感性划分，视频帧接收与解码、人脸模糊、YOLO26 检测、ByteTrack 追踪、特征提取、风险判定和事件摘要构建，都被纳入 SecureCore 的逻辑隔离边界。相对来说，认证、图表展示、事件列表、告警通知、权限管理和差分隐私统计服务，则运行在普通业务层。

这种边界划分有两个主要目的。第一，系统希望把原始视频和中间敏感特征尽量限制在受控处理链路内部。第二，系统也希望不要把不必要的模块放进可信

边界。这样可以同时兼顾安全性、可维护性和工程部署的可行性。

表3-1 模块当前运行位置与目标迁移位置划分

模块	当前运行位置	目标迁移位置	原因	当前状态
视频帧接收与解码	SecureCore 逻辑沙箱	secure_core / TEE 内	解码后即是完整原始画面，是最敏感的数据入口。	决赛增强设计
人脸模糊	SecureCore 逻辑沙箱	secure_core / TEE 内	匿名化前仍然包含可识别人脸与家庭环境信息。	当前原型已实现人脸模糊
YOLO26 检测与 ByteTrack 追踪	SecureCore 逻辑沙箱	secure_core / TEE 内	人体框、轨迹、位移虽低于原始帧敏感度，但仍可反推行为模式。	当前原型已实现
风险引擎与事件摘要构建	SecureCore 逻辑沙箱	secure_core / TEE 内	这是从低层特征转为高层事件的关键脱敏关口。	当前原型已实现，签名可作为增强项
认证、事件管理、告警、看板	普通业务层	普通业务层	不直接接触原始帧，只消费摘要结果。	当前原型已实现
差分隐私统计服务	普通业务层（发布侧）	普通业务层（发布侧）	只面向聚合统计输出，不进入实时识别主链路。	当前原型/决赛增强项

3.3 原型级完整性校验与远程证明迁移预留

在当前原型阶段，SecureCore 的启动过程采用了应用层完整性校验机制。具体来说，系统在启动时会分别对核心代码和模型文件计算哈希值，并生成 core_hash 和 model_hash。系统这样做，是为了检查关键组件是否发生了静态篡改。系统在校验通过后，才会加载模型并进入运行状态。之后，系统在生成事件摘要时，还会附带 core_hash、model_version 等字段。这样可以支持事件来源追

溯，也可以检查版本是否一致。

需要说明的是，上述机制属于原型级的可信启动校验。它主要用于本地部署场景中的防篡改和防篡改。当前系统还没有接入真实硬件 TEE 的远程证明服务，所以本系统不宣称已经实现了基于硬件根信任的标准 attestation 链路。

面向后续增强设计，系统已经提前预留了相关接口。现有的 measurement 字段、白名单校验逻辑和事件摘要凭证结构，都可以与未来的远程证明服务进行对接。这样一来，后续系统就可以在不重构业务核心的前提下，逐步接入标准化的可信执行环境证明框架，并实现从原型级完整性校验向硬件级可信证明的平滑过渡。

3.4 事件摘要设计

事件摘要是 SecureCore 与外部世界之间的最小接口。它的设计目标不是还原原始视频内容。它的主要目的是在满足告警处置、事件复核和统计分析需求的前提下，尽量减少那些可能被用来反向还原个体行为细节的信息。

基于这一原则，系统只输出一些必要字段，如表 3-2 所示。这些字段包括事件类型、风险等级、起止时间、匿名化人员标识、解释性特征摘要、脱敏截图引用和核心版本标识。系统不会输出完整原始轨迹、真实身份标识，也不会输出未经脱敏的画面内容。

这个机制和 SecureCore 的模块边界一起，构成了“受控处理—最小输出—分层使用”的数据保护路径。

表3-2 事件摘要字段定义

字段	说明	隐私特征
event_type / risk_level	事件类型与风险等级，用于后续告警与列表展示。	不包含原始图像内容
timestamp_start / end / duration	支持还原事件时段与持续时间。	保留业务必需时间信息
anon_person	由内部追踪 ID 映射出的匿名	阻断真实身份关联

_id	ID，如 p01、p02。	
feature_summary	中心点下落比例、长宽比变化、静止窗口时长等。	只保留解释性特征，不保存完整轨迹序列
blurred_snapshot_path	指向模糊后截图或短片段引用。	仅提供脱敏复核材料
core_hash / model_version	标记摘要由哪个可信核心与模型生成。	服务于完整性追溯

3.5 差分隐私统计发布机制

差分隐私只应该放在统计发布层，不应该加入实时识别链路。本系统将其定位为"图表和聚合结果的发布保护层"，适用接口包括每日事件数、每周高风险事件数、风险等级分布、夜间异常趋势、按社区组的聚合统计等。对这类计数型查询，采用拉普拉斯机制：对真实计数 c 输出 $c + \text{Lap}(\Delta/\epsilon)$ ，其中计数查询的敏感度 Δ 通常取 1。为兼顾可读性与隐私性，当前实现采用 $\epsilon = 0.8$ 的方案，并叠加低频值隐藏（低于阈值显示为"<阈值"）、负值截断（ $\max(0, \cdot)$ ）、短时缓存（10 分钟内重复查询返回同一扰动结果）与每日预算控制。

关键在于明确"作用边界"：实时告警、单条事件详情、家庭授权视图不应加噪，否则会直接损害照护效果。差分隐私适用于机构看板、平台运营分析、公共演示、科研接口等聚合统计发布场景，如表 3-3 所示。

表3-3 角色分级展示策略

角色 / 场景	可见数据	是否应加差分隐私	说明
家庭授权成员	实时告警、单条事件详情、模糊快照、单家庭统计	否	这类用户的目标是照护，不是做匿名统计；应优先保证可用性与准确性。
社区/机构值守人员	多对象事件摘要、班次看板、趋势图	视权限而定	单对象处置界面不加噪；跨对象聚合统计可开启轻量 DP。

平台运营/产品演示	匿名聚合图表、风险分布、趋势统计	是	防止从长期图表逆推出某位老人近期身体状况或生活规律。
科研接口 / 公共展示	跨家庭、跨房间、跨周期统计结果	是	该场景最应使用 DP，并限制重复查询与低频暴露。

3.6 系统整体架构与数据流

系统采用四层安全架构。四个层次分别是感知层、安全处理层、权威监督层和社区网络化服务平台。系统在整体设计上遵循一个基本原则。这个原则就是，原始视频尽量在受控边界内完成处理，普通业务层只使用脱敏后的结果。

在当前原型中，安全处理层由 **SecureCore** 逻辑隔离沙箱实现。它负责完成视频帧接收、解码、人脸匿名化、目标检测、多目标追踪、风险判定和事件摘要构建等敏感处理流程。权威监督层只接收匿名化后的事件摘要和脱敏复核材料。社区网络化服务平台只接收经过差分隐私保护后的聚合统计结果。

需要特别说明的是，当前系统中的安全处理边界仍然是软件级逻辑隔离边界，还不是已经完成部署和验证的硬件 TEE 边界。本作品的重点，不是宣称已经完成机密硬件落地。本作品更重视通过清晰的模块划分、最小接口暴露和可迁移的处理链路设计，为后续迁移到真实 TEE 或机密计算容器环境打下工程基础。

各层之间通过明确的接口协议通信。感知层与安全处理层通过 **WebSocket** 传输帧数据；安全处理层内部完成检测、追踪与风险判定后输出 **EventChange** 事件变更信号；权威监督层通过 **REST API** 向前端提供事件查询与告警管理接口；社区网络化服务平台通过差分隐私加噪后的统计接口对外发布聚合数据。

数据流分为两条路径。实时监控模式下，前端摄像头帧经 **WebSocket** 进入后端，系统采用“只处理最新帧”的跳帧策略降低队列堆积，再由检测器和风险引擎生成事件结果。离线视频模式下，用户主动上传视频用于评测或演示，后端在本地受控目录临时保存文件后进行逐帧处理。需要特别说明的是，“原始视频不落盘”应严格表述为：高保真视频流局限于沙箱内存处理与短时缓存，实时监控默认不长期保存原始流，分析闭环后物理销毁；离线视频评测在本地受控目录临时存放并可清理。

前端展示层基于 Vue 3、Vite、Element Plus 与 ECharts 实现，遵循"少看原始画面，多看事件与状态"的设计原则。前端面向家庭用户、平台用户、管理员三类角色提供多视图页面：家庭用户可访问 Dashboard 事件看板、Monitor 实时摄像头检测、VideoDetect 视频文件检测、Analysis 统计分析、System 系统设置；平台用户通过 CommunitySelect 选择社区组后进入 PlatformDashboard 查看聚合统计；管理员使用 AdminPlatform 进行平台管理；Visualization 可视化大屏页面展示实时监控画面；Login 登录页面实现用户认证与 JWT 令牌管理。

后端以 Quart 作为异步 Web 框架，注册 auth、detect、events、alerts 与 platform 等业务蓝图。数据持久化层使用 SQLite + SQLAlchemy。JWT 令牌配合 bcrypt 密码哈希实现用户认证，令牌通过 Authorization 头传递，过期时间 24 小时。

3.7 行为识别、追踪与风险分级

3.7.1 人体姿态检测

系统采用 YOLO26 深度学习模型实现人体姿态检测，以 ONNX 格式部署于服务端。模型输入为单帧 RGB 图像，经预处理流程：首先将原始帧缩放至 320×320 分辨率，转换色彩空间从 BGR 到 RGB，调整数据排列为通道优先格式（CHW），归一化至 0-1 浮点范围，最后添加批次维度形成四维输入张量。

模型推理输出原始张量形状为 [1, 6, 2100]，其中 2100 为预设锚点数量，每个锚点对应 6 维向量：前四维为边界框参数（中心点坐标 cx、cy，宽高 w、h），后两维为类别概率分数（class0_score 对应站立姿态，class1_score 对应跌倒姿态）。后处理流程首先对输出张量进行转置，得到 [2100, 6] 格式；使用 argmax 提取各锚点的最高置信度类别；应用置信度阈值过滤低质量候选，默认阈值 0.3；使用 NMS 非极大值抑制去除重叠框，阈值 0.45；按置信度降序排序，最多保留 3 个检测结果。类别标签映射规则为：类别 0 标记为 "normal" 表示站立或正常活动状态，类别 1 标记为 "fallen" 表示跌倒姿态。

3.7.2 隐私保护机制

系统在检测流程中集成人脸隐私保护模块，采用 MediaPipe 人脸检测模型实现人脸定位，配合高斯模糊实现人脸区域匿名化。检测到人脸后，系统执行区

域扩展与模糊处理：首先计算人脸区域中心点坐标，按 50% 比例扩大模糊区域范围，确保覆盖面部周边特征；然后使用 51×51 核大小的高斯模糊滤波器对扩展区域进行处理，模糊强度足以掩盖面部特征细节。隐私保护模块在检测输出帧之前执行，确保前端接收的图像帧中人脸已被匿名化处理。

当 MediaPipe 偶尔漏检时，系统通过 ByteTrack 追踪 ID 的相对比例推断人脸位置继续模糊，超过 30 帧丢失才放弃追踪，保障匿名化的连续性。

3.7.3 多目标追踪

系统采用 ByteTrack 算法实现跨帧多目标追踪，为每个检测到的个体分配唯一追踪 ID，支持多人同时追踪场景。ByteTrack 追踪器参数配置：检测阈值 `track_thresh=0.3` 用于初始化新轨迹，匹配阈值 `match_thresh=0.8` 用于帧间轨迹关联，最大丢失帧数 `max_age=30` 定义轨迹保持期限，最小确认帧数 `min_hits=1` 决定轨迹确认条件。

追踪流程每帧执行：将当前帧检测结果组织为 `[x1,y1,x2,y2,conf,cls]` 格式输入追踪器；追踪器基于运动预测与外观相似度进行帧间匹配，为每个检测框分配追踪 ID；对于短暂丢失的目标，追踪器在 `max_age` 帧内保持轨迹状态，避免频繁 ID 切换。

位移计算基于边界框中心点的欧氏距离。每帧记录每个追踪 ID 的边界框中心坐标，计算当前帧与上一帧中心点的像素距离作为 `movement` 值，用于后续静止状态判定。

3.7.4 风险分级引擎

如图 3-1 所示，首先，系统接收视频帧输入。然后，视频帧进入 YOLO26 检测模块。这个模块会生成人体目标框、类别标签和置信度。在此基础上，ByteTrack 会对检测结果进行跨帧追踪。系统会为每个目标分配稳定的 ID。系统也会计算相邻帧之间的人体位移量。接着，RiskEngine 会根据逐帧检测结果、持续时间和运动特征，对跌倒、长时间静止和夜间异常等事件进行综合判定。然后，系统会输出对应的风险等级。最后，系统形成 `PersonRisk` 结果。这个结果可以为后续的事件记录、告警触发和风险分级响应提供依据。整体来看，这条流程体现了“检测—追踪—判定—输出”的核心逻辑。它也是系统实现异常行为识别和实时风险评估的关键数据链路。

异常行为识别与风险判定数据流总览图

Overview of Data Flow for Abnormal Behavior Recognition and Risk Assessment

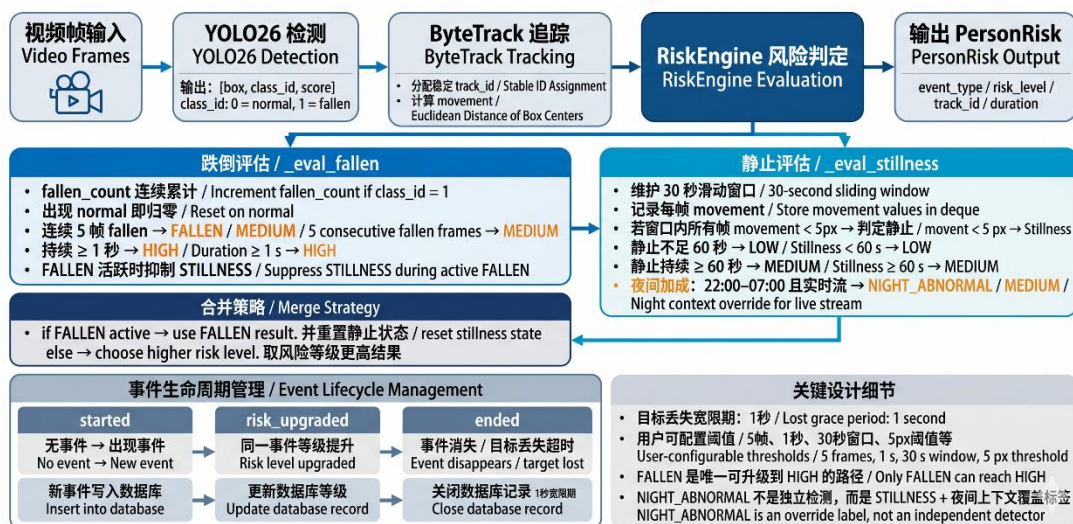


图 3-1 异常行为识别与风险判定数据流总览图

风险引擎基于检测层输出的逐帧数据，执行跨帧状态分析与风险等级判定。系统定义四级风险体系：NORMAL 为正常状态，无异常事件；LOW 为低风险状态，提示潜在异常；MEDIUM 为中等风险状态，需要关注；HIGH 为高风险状态，需要紧急响应。

跌倒风险判定规则：当检测到 fallen 类别时启动跌倒事件判定；连续 5 帧检测为跌倒姿态后确认跌倒事件，风险等级升至 MEDIUM；跌倒状态持续超过 1 秒后风险等级升至 HIGH，触发紧急响应机制。跌倒恢复站立姿态时立即重置状态计数器。

静止风险判定规则：系统维护 30 秒滑动窗口记录每帧位移值；当窗口内所有帧的 movement 值均低于 5 像素阈值时判定为静止状态；静止持续 30 秒后触发 LOW 风险等级；静止持续 60 秒后升级至 MEDIUM 风险等级。窗口使用计数器优化，记录超出阈值的帧数量，避免全窗口遍历。

夜间异常风险判定规则：系统定义夜间时段为 22:00 至 07:00；当实时摄像头检测流在夜间时段触发静止判定时，风险等级直接定为 MEDIUM，标记为夜间异常事件。风险引擎支持用户自定义参数配置，包括跌倒确认帧数、静止阈值、夜间时段等，参数配置存储于数据库，按用户维度隔离。

3.7.5 通信协议与数据流

视频文件检测流程：前端通过 HTTPPOST 接口上传视频文件，后端生成唯

一 video_id 并存储文件；前端建立 WebSocket 连接至 /ws/video/<video_id>；后端逐帧读取视频、执行检测、绘制风险框、编码为 JPEG 帧；通过 WebSocket 发送帧结果与进度信息；视频处理完成后发送汇总结果。

实时摄像头检测流程：前端调用 POST /api/session/create 创建检测会话；前端启动摄像头采集，通过 WebSocket 连接至 /ws/detect/<video_id> 发送帧数据；后端采用跳帧策略处理，优先处理队列中的最新帧，丢弃积压的旧帧以保持实时性；检测结果实时返回前端展示；会话结束时调用 POST /api/session/close 关闭并持久化事件记录。

事件持久化机制：风险引擎输出 EventChange 事件变更信号，包括 started（事件开始）、risk_upgraded（风险升级）、ended（事件结束）三种类型。事件变更实时写入数据库 Event 表，记录事件类型、风险等级、开始时间、结束时间、帧计数等信息。告警服务监听 ended 事件，针对 HIGH 和 MEDIUM 等级触发告警通知。

3.8 设计重点与难点

表3-4 设计重点与难点分析

设计重点 / 难点	难点说明	解决思路
隐私与可用性的平衡	如果所有数据都匿名到无法使用，系统就失去照护价值；但如果完全原样暴露，又失去项目立意。	将原始视频保护、事件摘要输出和统计发布分层处理，允许授权家属查看事件，限制公共视图仅看匿名聚合。
实时性与准确性的平衡	本地服务器算力有限，多阶段处理容易积压 WebSocket 队列。	使用 ONNX 轻量化推理、跳帧策略和“只处理最新帧”的队列清空机制。
误报抑制	躺卧、弯腰、遮挡等姿态可能与跌倒相似。	用连续帧确认、持续时间升级和多源特征（长宽比、中心点变化、静止时长）共同判定。
多目标追踪	人物短暂遮挡或离画	调整 ByteTrack 参数，并在

稳定性	时，追踪 ID 易切换。	风险引擎引入短时宽限，避免过早结束事件。
通用算力平台下的可信边界构建问题	真实硬件级 TEE 依赖专用底层支持，在当前原型验证阶段难以直接获取并完成全链路部署测试	设计并实现“软硬解耦的 SecureCore 逻辑沙箱”。在逻辑控制流上完全模拟硬件 TEE 的边界防御和完整性校验，兼顾了当下的计算设备泛用性与未来向专用保密计算节点迁移的零成本扩展需求。
差分隐私的适用边界	很多系统误把 DP 用到实时告警或单条详情，导致结果不可用。	明确规定 DP 只作用于 /stats 类聚合发布接口，并采用角色分级展示。

3.9 真实性说明

本作品当前版本还没有在 SGX、TrustZone、Open Enclave 或机密虚拟机等真实硬件 TEE 环境中完成部署和性能测试。现阶段已经完成的，是面向 TEE 迁移的 SecureCore 逻辑隔离原型。系统目前已经完成了敏感链路封装、应用层完整性校验、事件摘要最小暴露和分层隐私发布等功能验证。至于硬件级内存隔离、硬件根信任和标准远程证明等能力，目前还没有在本作品中正式实现。这些内容属于后续的增强设计和迁移目标。

第4章 系统实现

本章从工程实现角度，详细阐述第 3 章技术方案的具体落地过程，包括软件设计、用户界面、数据来源、模型训练、开发迭代中的改进与问题解决。

4.1 系统整体架构

本系统面向独居老人居家安全监护场景。系统围绕“隐私优先、实时识别、

分级响应、统计发布分层保护”这一目标进行设计。系统整体由感知采集层、安全处理层、家属监护端和社区或网格化养老平台组成。系统在总体设计上遵循一个基本原则。这个原则就是，原始视频尽量在受控边界内完成处理，外部模块只获取与自身角色相匹配的数据结果。系统希望在保证异常行为识别和应急响应能力的同时，尽量降低原始视频和行为细节向系统外部扩散所带来的隐私风险。

从总体功能划分来看，系统主要由三条链路组成。这三条链路彼此协同，但边界比较清楚。第一条链路是实时监测链路。它负责完成从视频输入、异常识别、风险判定、事件摘要生成，到家属侧实时告警和处置的全过程。第二条链路是事件记录链路。它负责把经过脱敏处理后的事件结果写入数据库，用来支持事后查询、事件复核和授权查看。第三条链路是统计发布链路。它负责对历史事件记录进行聚合分析，并在差分隐私保护下，向平台侧输出日报、周报、风险趋势和网格统计等统计结果。

系统整体架构如图 4-1 所示。这三条链路在逻辑上彼此衔接，但它们在数据权限和隐私保护强度上是逐层递进的。通过这样的设计，系统形成了“实时处置优先、统计发布受控”的分层架构。

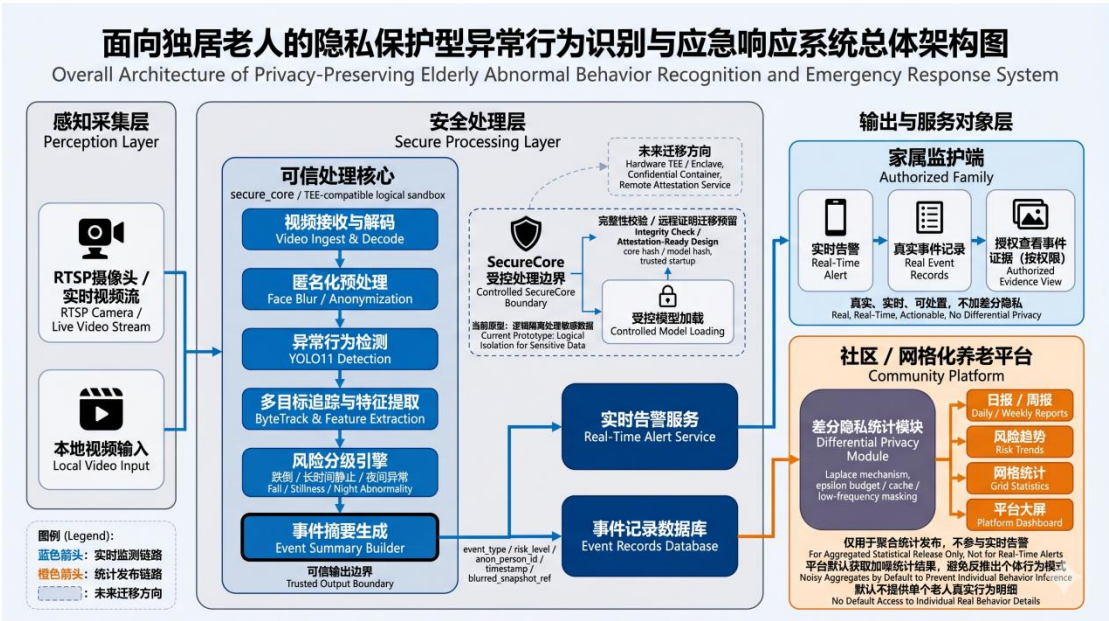


图4-1 系统整体架构

4.2 数据来源与数据集准备

系统采用 LE2I 公开跌倒检测数据集作为训练数据来源。该数据集包含多个物理场景下的跌倒视频，涵盖不同摄像头视角、光照条件和跌倒姿态。原始标注

以帧号为粒度，通过 `fall_start` 和 `fall_end` 标记每段视频中跌倒事件的起止时间。

数据集共约 31,000 张图片，划分为二分类检测任务（`normal` 和 `fallen`）。本项目严格按场景隔离划分，将不同的物理环境场景分别分配至训练集、验证集和测试集，比例约为 4:4:2。与传统的 7:2:1 随机划分不同，这种方式确保验证集和测试集中的场景在训练阶段从未出现过，防止同一场景的帧同时出现在训练集和测试集中导致数据泄露，避免模型通过记忆背景而非学习跌倒特征来“作弊”，从而保证评估结果能够真实反映模型的跨场景泛化能力。

类别标注基于原始帧号范围生成：帧号处于跌倒事件区间内（`fall_start` 至 `fall_end`）时标注为 `fallen`，其余帧标注为 `normal`。标注结果以 YOLO 格式保存，供模型直接训练使用。

4.3 模型训练与迭代优化

初始方案设计为三分类检测（`normal/falling/fallen`），期望模型能区分跌倒过程与跌倒结果。实际训练后发现 `falling` 类样本数量极少，且该类别对应的是短暂的过渡帧，模型难以有效学习该类别特征。

由于 LE2I 等开源数据集在原始标注时，对长视频中“`falling`”（跌倒过渡期）区间的边界界定存在较大人工模糊性，导致模型在过渡态往往会产生高频判定震荡。为此，项目组提出了一种对抗长尾标注误差的“1:9 动态切割策略”：平滑剥离特征重合度极高的 `falling` 类别，将原区间前 10%（身体尚处直立倾斜阶段）强制对齐至 `normal`，而将后 90%（质心已剧烈下坠失去姿态平衡）统一归集为 `fallen`。基于质心物理规律重构的标签库大幅消除了样本级的模棱两可，极大地增强了模型在判定跌倒临界点时的抗扰动鲁棒性。

训练参数配置：选用 `yolov11s` 作为基础模型（兼顾精度与速度），输入图像尺寸 320×320 ，训练轮数 100，批次大小 32，早停耐心值 20 轮，学习率采用默认预热策略。训练使用 NVIDIA GPU 加速。模型权重保存为 `best.pt` 文件，并通过 ONNX 导出工具转换为 ONNX 格式（`opset=12`、输入尺寸 320×320 ），部署至 SilentElderSense 后端系统。模型的详细评估指标与各维度测试结果见第 5 章测试分析。

4.4 后端软件设计与模块实现

后端采用 Quart 异步框架，支持 `async/await` 语法处理并发请求。项目结构按业务职责划分为四个模块。

`auth` 认证模块实现用户注册、登录、JWT 令牌生成与验证。密码使用 `bcrypt` 哈希存储，令牌有效期 24 小时，通过 `Authorization` 头传递。模块包含 `routes.py` 路由定义、`models.py` 数据库模型、`utils.py` 令牌工具函数。

`detect` 检测模块是核心业务模块，实现视频文件上传接口、`WebSocket` 视频处理接口、实时摄像头检测接口、检测参数配置接口。模块集成 `FallDetector` 检测器与 `RiskEngine` 风险引擎，处理流程包括帧解码、检测推理、风险判定、结果编码、`WebSocket` 推送。`routes.py` 包含完整的 `WebSocket` 处理逻辑，支持跳帧策略保持实时性。

`events` 事件模块提供事件记录的 `CRUD` 接口，支持按时间范围、风险等级、事件类型筛选查询，返回分页结果。`models.py` 定义 `Event` 数据模型，包含用户 ID、视频 ID、人员 ID、事件类型、风险等级、开始时间、结束时间、帧计数等字段。

`alerts` 告警模块实现三级响应服务框架，根据风险等级自动触发本地提醒、管理端告警与紧急通知。短信、邮件等外部通知通道可在实际部署时对接第三方服务接入。数据库采用 `SQLite` 轻量级存储，配合 `SQLAlchemy` ORM 实现数据持久化。数据库文件存储于 `backend/data/db.sqlite3`，首次启动自动创建表结构。

4.5 检测核心模块实现

检测核心封装为独立 Python 模块，位于 `backend/secure_core` 目录。`FallDetector` 类是主检测器，初始化时加载 `ONNX` 模型，提供 `create_session`、`process_frame`、`close_session` 三个核心接口。`process_frame` 方法执行完整检测流程：预处理缩放与归一化、`ONNX` 推理、后处理 `NMS`、人脸模糊处理、追踪 ID 分配、位移计算。`SessionManager` 类是会话管理器，封装 `ByteTrack` 追踪器，维护每个会话的追踪状态与位移历史。每帧更新追踪结果，计算边界框中心点位移，清理过期会话状态。`types` 模块定义数据结构类型，包括 `FrameResult` 单帧结果、`PersonResult` 单人结果、`SessionResult` 会话结果，使用 `dataclass` 简化类型定义。

风险引擎封装于 `backend/detect/risk_engine.py`。`RiskEngine` 类是核心风险判

定器，提供 `create_session`、`process`、`close_session` 三个接口。`process` 方法接收每帧检测结果，执行跌倒连续帧判定、静止滑动窗口判定、夜间时段判定，返回每个人的风险等级与事件变更信号。`PersonRiskState` 类维护单人跨帧状态，包括跌倒计数器、静止滑动窗口、事件开始时间戳等状态数据。`EventChange` 类表示事件变更信号，包含变更类型（`started/risk_upgraded/ended`）、人员 ID、事件类型、风险等级、时间戳、帧计数等信息，用于持久化事件记录。

4.6 前端软件设计与用户界面

前端基于 Vue 3 框架开发，采用 Composition API 组织组件逻辑，Element Plus 组件库提供统一 UI 风格。前端实现多个功能页面，涵盖实时监控、事件看板、统计分析、系统设置与平台管理等核心功能。

Login 登录界面如图 4-2 所示，页面提供简洁的登录表单，支持用户名密码登录，JWT 令牌存储于 Pinia 状态管理，登录成功后跳转 Dashboard。

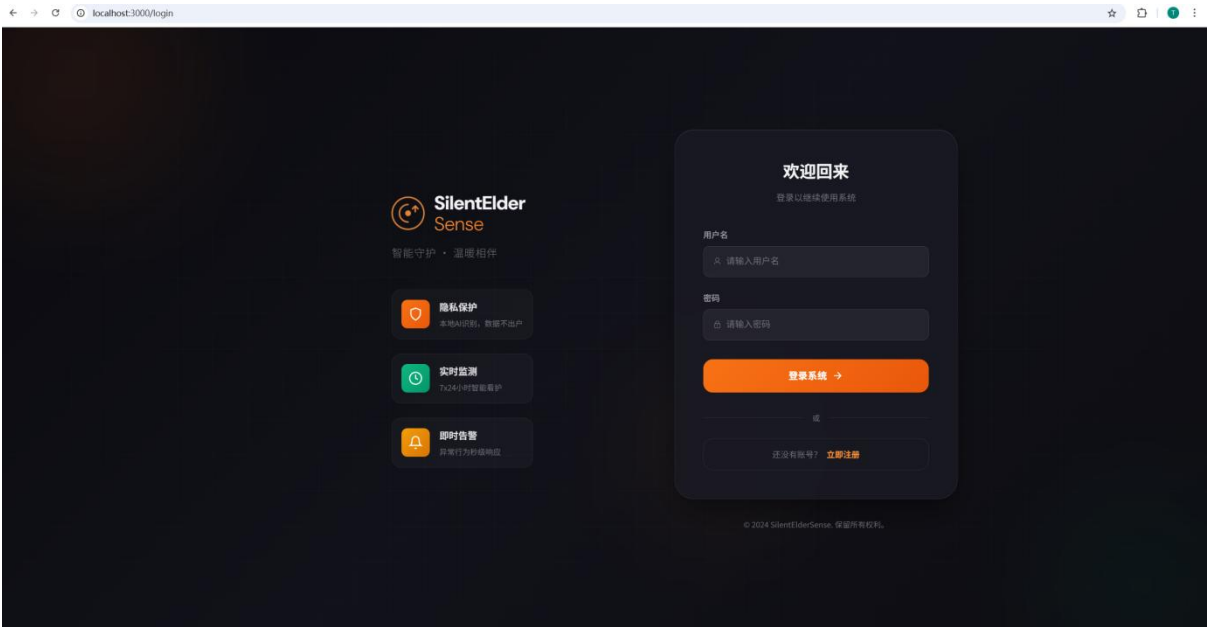


图4-2 Login登录界面

Dashboard 事件看板界面如图 4-3 所示，页面顶部展示四个统计卡片，包含总事件数、高风险事件、待处理事件、本周新增；中部展示两个饼图，分别为事件类型分布和风险等级分布；底部展示柱状图，呈现按日期的事件数量趋势。

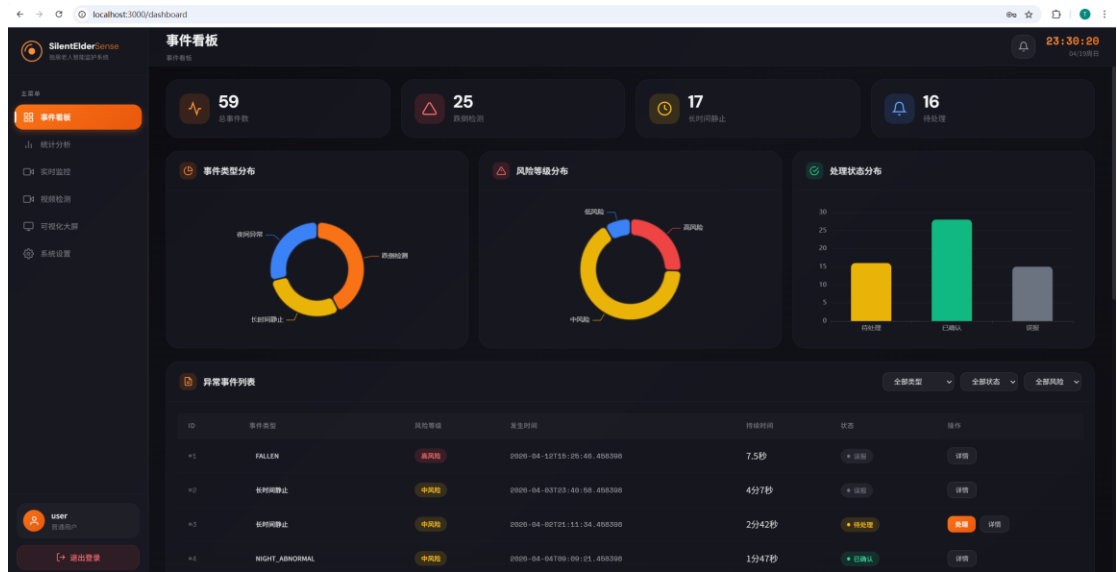


图4-3 Dashboard事件看板界面

Monitor 实时摄像头检测界面如图 4-4 所示，页面支持摄像头设备选择与刷新、检测启停控制、实时帧显示与风险框绘制、会话状态与延迟信息展示。页面采用 16:8 布局，左侧为视频画布，右侧为检测结果详情面板。

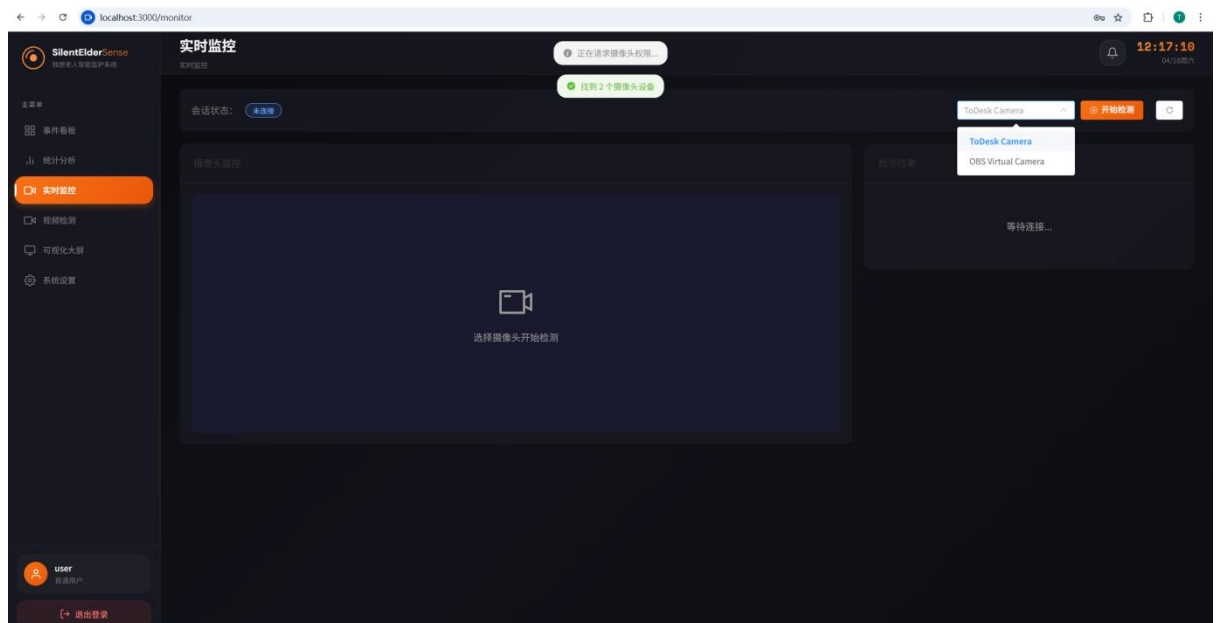


图4-4 Monitor实时摄像头检测界面

VideoDetect 视频文件检测界面如图 4-5 所示，页面支持视频文件拖拽上传或点击选择，上传后自动建立 WebSocket 连接开始处理，实时显示处理进度与帧检测结果，处理完成后展示汇总统计。

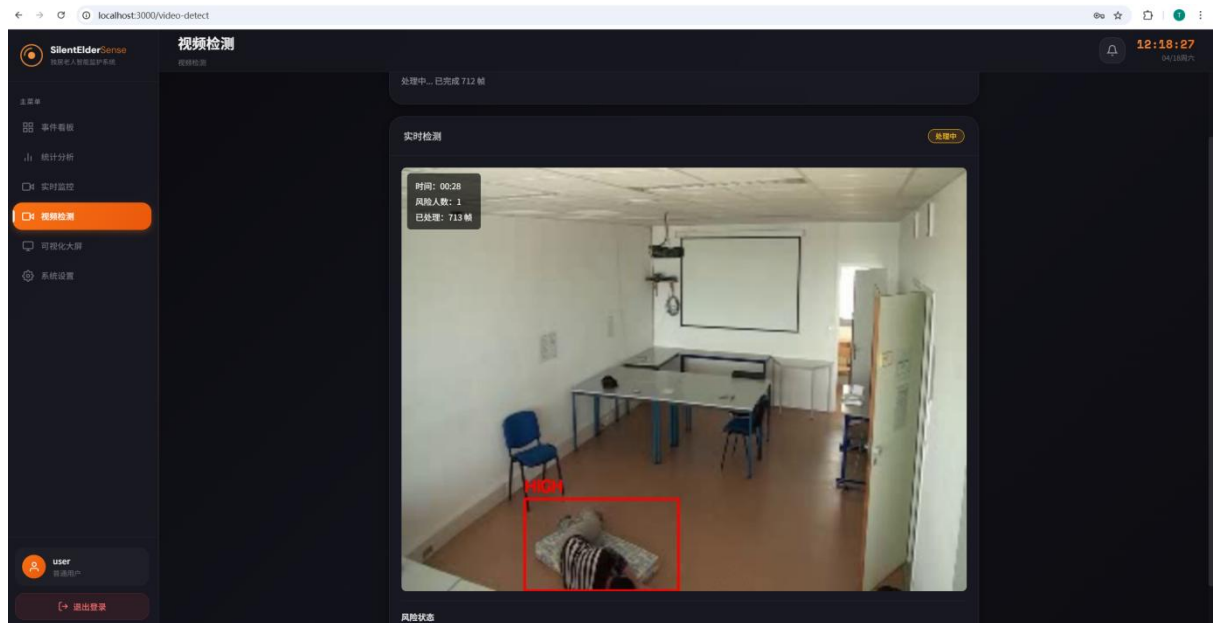


图4-5 VideoDetect视频文件检测界面

Analysis 统计分析界面如图 4-6 所示，页面支持按时间范围筛选事件记录，表格展示事件列表，包含时间、类型、风险等级、状态，提供分页与排序功能。

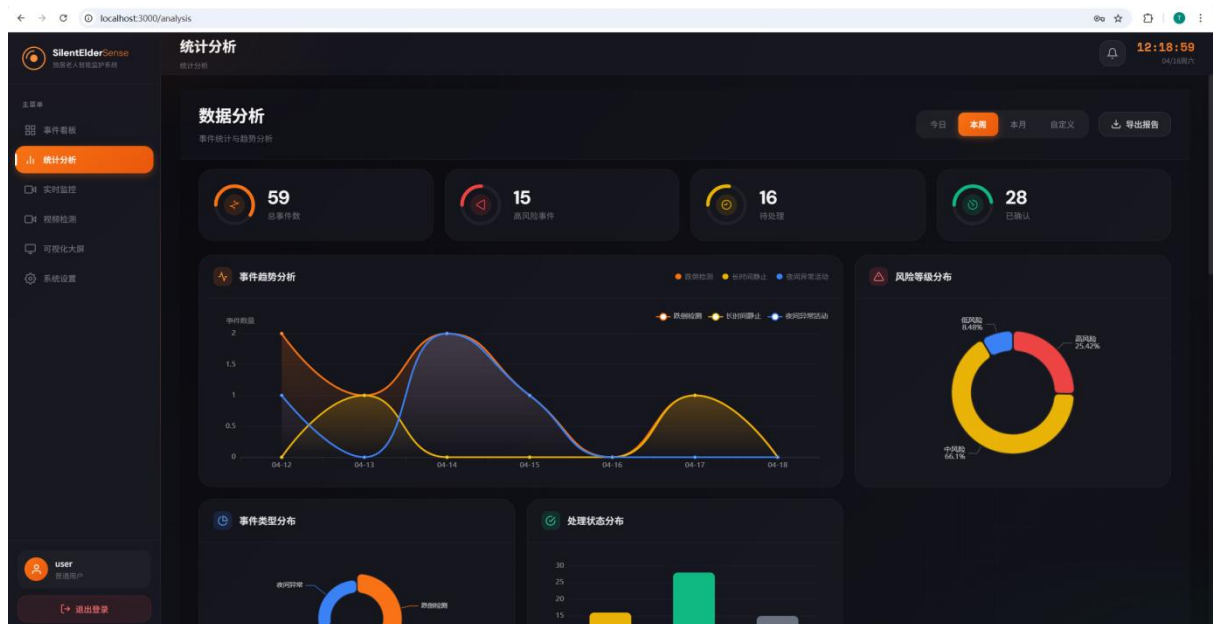


图4-6 Analysis统计分析界面

System 用户系统设置界面如图 4-7 所示，页面提供检测参数配置表单，可调整跌倒确认帧数、跌倒升级秒数、静止窗口时长、静止位移阈值、静止升级秒数、夜间时段起止小时等参数，配置保存后实时生效。

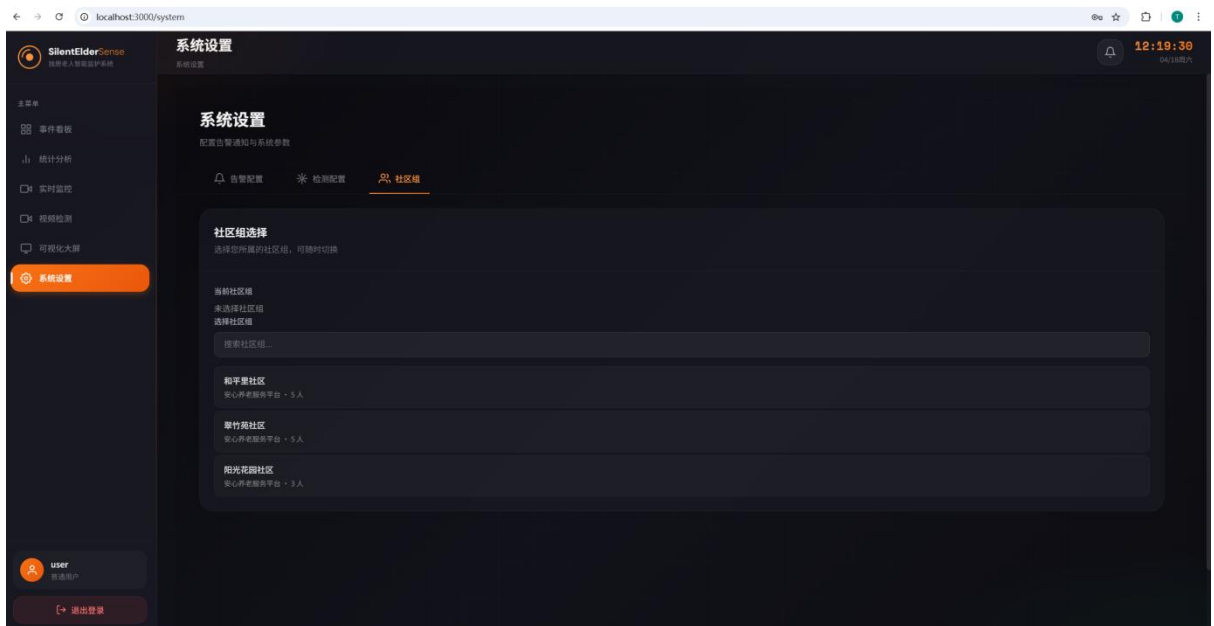


图4-7 System用户系统设置界面

AdminPlatform 平台管理界面如图 4-8 所示，管理员登录后可进入平台管理页面，对系统用户、社区组织进行统一管理与权限分配。

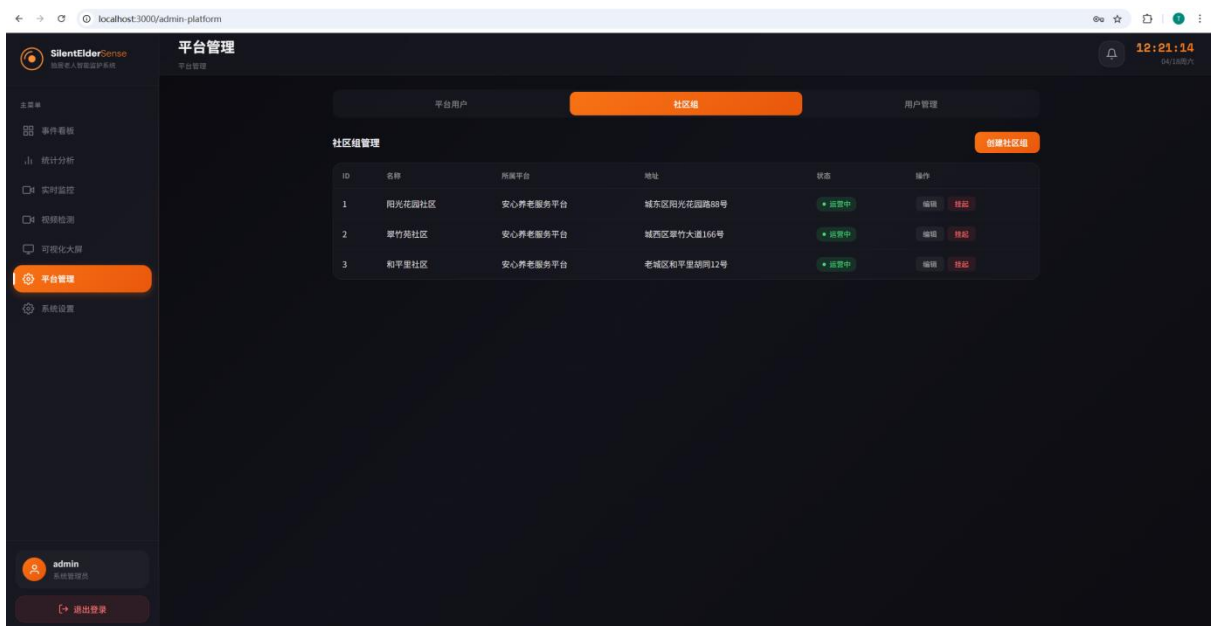


图4-8 AdminPlatform平台管理界面

PlatformDashboard 平台分析界面如图 4-9 所示，平台用户登录后进入平台分析页面，可查看所辖社区的统计数据。该页面的统计查询经差分隐私层加噪处理后返回，确保个体隐私不被泄露。

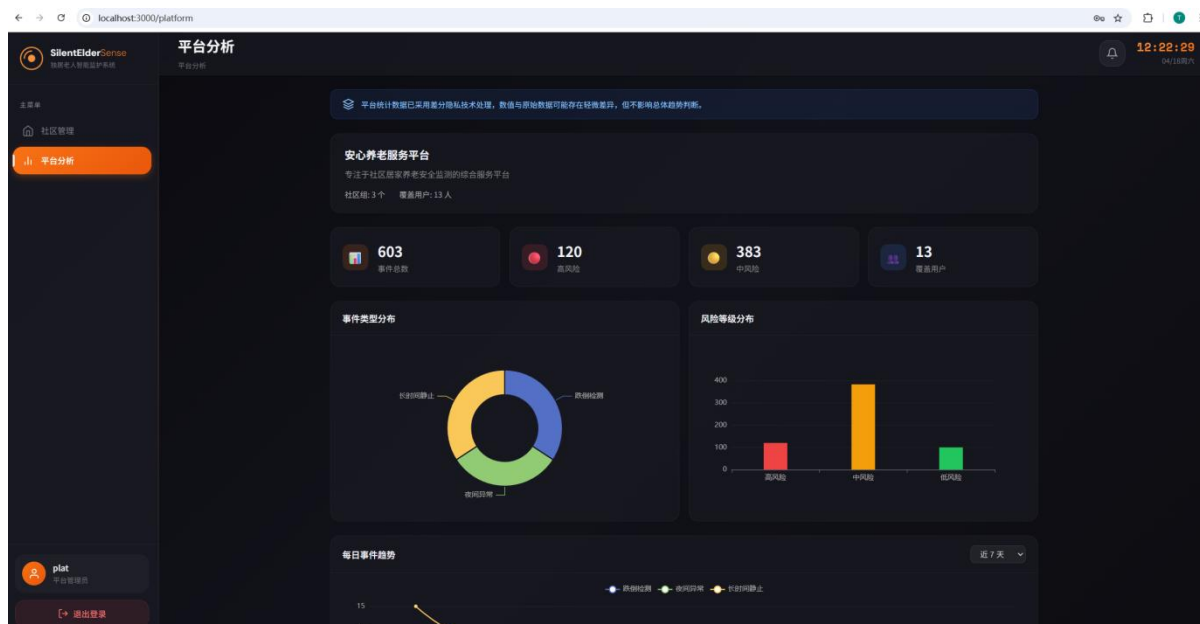


图4-9 PlatformDashboard平台分析界面

Visualization 可视化大屏界面如图 4-10 所示，页面采用全屏展示模式，左侧为实时监控画面，右侧为统计图表与告警列表，适合值班监控场景使用。

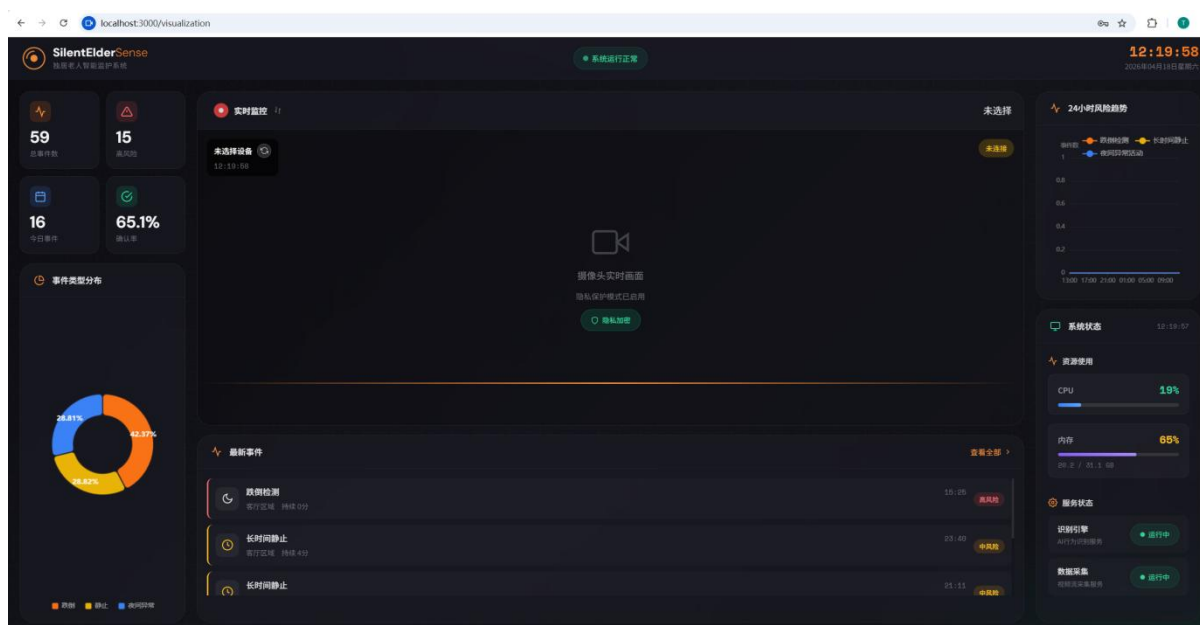


图4-10 Visualization可视化大屏界面

4.7 系统部署方法

为适应真实的异构计算环境与适老场景的本地化部署需求，本系统全面采用了轻量级容器化与边缘/中心平台解耦的生产级部署方案。

(1) 本地服务器（后端服务）部署：在实际落地中，通过 Docker 容器化技

术对底层环境依赖（如 CUDA、ONNX Runtime 等）进行标准封装，从而屏蔽不同微型计算设备的操作系统差异。生产环境下采用 Gunicorn + Uvicorn 构建基于 ASGI 的高性能工作进程网关，接管核心 API 请求，从容处理高并发的 WebSocket 视频流。同时配合挂载的高内聚 SQLite 持久化卷，有效防范网络闪断时的内存状态与事件丢失。

（2）平台管理节点（前端服务）部署：平台前端工程脱离开发态服务器，由自动化流水线执行前置构建产出高度优化的纯静态压缩资源包（dist 包）。中心平台主节点依托高性能 Nginx 反向代理服务器承载静态发版，同时作为前置网关负责底层 API 及 WSS（WebSocket Secure）的流量回源。前后端分离架构已预留统一的网关配置层，在实际部署时可快速挂载 Nginx 反向代理并配置 HTTPS 安全信道，从容应对生产环境的传输加密需求。

（3）模型部署与在线更新：在后端边缘节点预设的 models/ 热挂载目录下，可直接注入基于本数据集精调的 YOLO26s.onnx。该容器级别的路径映射模式支持系统在不切断主服务进程的情况下零停机（Zero Downtime）实现推理算法的平滑迭代升级。

4.8 开发中遇到的困难与解决方法

WebSocket 实时帧传输延迟问题。初始方案中，前端持续发送帧数据，后端逐帧处理并返回结果。当网络波动或处理速度跟不上时，WebSocket 消息队列积压导致延迟累积。为此，我们在路由端实现了一套基于异步探针的非阻塞跳帧调度机制：当系统处理吞吐达不到网络推流速度时，自适应丢弃队列中的陈旧帧，优先抓取最新帧推入检测网络。实践证明，这套并发抛弃逻辑不伤主进程性能，让系统能在恶劣网络和高频积压场景下，依旧保障核心检测链路的低延迟响应与实时告警能力。

跌倒类别样本不平衡问题。原始数据集中 falling 类样本仅 52 张，占总数约 0.2%，模型难以有效学习该类别。解决方法是将三分类简化为二分类，调整 falling 区间的标注策略，使过渡帧也能稳定输出检测框。

人脸检测模型选择问题。尝试了多种人脸检测方案，包括 Haar Cascade、SCRFD 等，发现 Haar Cascade 对侧脸遮挡情况检测效果不佳，SCRFD 计算开销较大。解决方法是采用 MediaPipe 人脸检测模型，基于深度学习实现，检测速度快、准确率高，对各种姿态具有较好的鲁棒性。

追踪 ID 频繁切换问题。当人员短暂离开画面或被遮挡时，ByteTrack 追踪器可能丢失轨迹并重新分配新 ID，导致状态统计混乱。解决方法是调整追踪器参数 `max_age=30`，允许 30 帧内的丢失保持原有轨迹；同时在风险引擎中实现宽限期机制，人员消失后保持 1 秒状态缓存，避免频繁的事件结束信号。

风险框绘制色彩一致性问题。检测结果需在帧上绘制风险等级标识框，不同等级使用不同颜色。解决方法是定义 `RISK_COLORS_BGR` 颜色映射字典，HIGH 等级红色、MEDIUM 等级黄色、LOW 等级浅蓝、NORMAL 等级绿色，前端接收处理后的帧图像直接显示，无需二次渲染。

第5章 测试分析

本章将从四个核心方面对系统进行测试和验证。

第一，本章将测试异常行为识别效果。系统将重点检验跌倒检测模型的精度、召回率和实时性。

第二，本章将测试风险分级和规则引擎。系统将重点检验多级风险判定逻辑是否正确，也会检验系统运行是否稳定。

第三，本章将测试 SecureCore 逻辑隔离原型。系统将重点检验敏感链路封装、完整性校验和事件摘要最小暴露等机制是否能够正常发挥作用。

第四，本章将测试差分隐私统计发布机制。系统将重点检验拉普拉斯加噪、预算控制和低频隐藏等发布保护方法是否有效。

需要特别说明的是，本章关于 SecureCore 的测试，检验的是面向 TEE 迁移的逻辑隔离原型能力。当前测试并不针对真实硬件 TEE 环境下的内存隔离、硬件根信任或标准远程证明能力。换句话说，本章测试结果主要说明的是，在普通 CPU 环境下，系统已经能够实现敏感处理链路封装、摘要最小暴露和原型级完整性校验。同时，这些结果也说明，系统已经具备了向未来机密计算环境平滑迁移的工程基础。

5.1 测试目标与测试环境

如表5-1所示，本系统的测试环境为 Windows 11 操作系统下的普通 x86 CPU 平台。系统通过 ONNX Runtime XNNPACK delegate 运行模型推理。系统前端和后端分别部署在 localhost:3000 和 localhost:8000。这个环境主要用于验证系统在非专用机密硬件条件下的功能正确性、实时性，以及 SecureCore 逻辑隔离原型的可行性。需要说明的是，当前测试环境不包含 SGX、TrustZone、Open Enclave 或机密虚拟机等真实硬件 TEE 支持。所以，本章不会对硬件级机密计算性能作出声明。

表5-1 测试环境配置

项目	配置
操作系统	Windows 11 Pro
GPU	无独立显卡环境（标准多核 CPU）
推理引擎	ONNX Runtime 1.24.4 (CPU & XNNPACK)
模型	基于本数据集精调的 YOLO26s
输入尺寸	320 x 320
前端	localhost:3000 (Vue 3)
后端	localhost:8000 (Quart)

表 5-2 统计了训练集、验证集和测试集的图像数量、目标数量以及类别分布情况。整个数据集采用了 normal 和 fallen 两类标注方式。其中，测试集中 fallen 类样本数量相对较少，这也反映了异常事件在真实场景中本来就比较少。

表5-2 测试数据集概况

数据集	图像数	目标数	类别分布
训练集	11827	11843	normal / fallen
验证集	13192	13216	normal 7925, fallen 5291
测试集	6116	6116	normal 4805, fallen 1311

5.2 异常行为识别效果测试

5.2.1 跌倒检测效果

如表 5-3 所示，使用训练好的 YOLO26s 精调模型在验证集和测试集上进行评估，采用 Precision、Recall 和 mAP50 作为评价指标。验证集包含 13192 张图像，测试集包含 6116 张图像。

表5-3 跌倒检测模型评估结果

	Precision	Recall	mAP50
Train	0.914	0.911	0.959
Val	0.860	0.865	0.934
Test	0.776	0.817	0.827

模型在训练集上 mAP50 达到 0.959，验证集 mAP50 为 0.934，测试集 mAP50 为 0.827。整体性能表现良好，满足部署需求。推理速度：预处理 0.3ms、推理 4.9-5.1ms、后处理 0.9ms，满足实时性要求。

5.2.2 长时间静止检测

如表 5-4 所示，当人物在 30 秒窗口内运动量低于阈值时触发 STILLNESS LOW 事件，持续超过 60 秒升级为 MEDIUM。

表5-4 长时间静止检测规则配置

参数	值	说明
STILLNESS_WINDOW_SECONDS	30	静止判定观测窗口
STILLNESS_MOVEMENT_THRESHOLD	5.0	运动量阈值
STILLNESS_ESCALATE_SECONDS	60	升级为 MEDIUM 的时间
触发条件	实时流+连续静止	仅在实时监控流中触发

5.2.3 夜间异常检测

如表 5-5 所示，22:00 至次日 7:00 期间检测到人物静止时触发 NIGHT_ABNORMAL 事件，风险等级 MEDIUM。该事件参数可自定义配置。

表5-5 夜间异常检测规则配置

参数	值	说明
NIGHT_START_HOUR	22	夜间起始时间
NIGHT_END_HOUR	7	夜间结束时间
触发条件	夜间 +STILLNES S+实时流	三者同时满足
风险等级	MEDIUM	夜间静止事件等级

5.3 风险分级与规则引擎验证

如表 5-6 所示，通过构造模拟帧数据传入 RiskEngine，验证帧数确认、时间升级和恢复检测机制。

表5-6 风险分级与规则引擎测试结果

测试项	输入条件	预期输出	实际输出	通过
跌倒确认(4 帧)	连续 4 帧 fallen	无事件	无事件	是
跌倒确认(5 帧)	连续 5 帧 fallen	FALLEN MEDIUM	FALLEN MEDIUM	是
跌倒升级(大于 1s)	fallen 持续大于 1 秒	FALLEN HIGH	FALLEN HIGH	是
恢复活动	5 帧 fallen 后 3 帧 normal	事件结束	事件结束	是

帧数确认机制 (FALLEN_CONFIRM_FRAMES=5) 和时间升级机制 (FALLEN_ESCALATE_SECS=1.0s)均按预期工作。连续 4 帧不误触发，5 帧准确触发 MEDIUM，持续 1 秒升级 HIGH，恢复后事件结束。

5.4 SecureCore 逻辑隔离与完整性校验测试

5.4.1 架构边界验证

本测试主要用于验证 SecureCore 原型是否已经实现了对敏感处理链路的受控封装。测试结果表明，模型推理、风险判定和事件摘要构建等核心处理流程都在 SecureCore 内部完成。普通业务层和前端接口不会直接暴露原始检测结果、原始追踪标识或完整行为轨迹。这个结果说明，当前原型已经形成了比较清晰的“敏感处理边界—脱敏结果输出”结构。这种结构有助于系统在普通部署环境下，先验证受控处理和最小暴露原则是否能够成立。

需要说明的是，这项测试验证的是软件级逻辑隔离边界是否有效。它并不等同于真实硬件 TEE 在物理内存隔离、总线保护或侧信道对抗方面的安全能力。所以，本作品在这里不宣称已经完成硬件级机密计算验证。本作品更强调的是，当前原型已经具备了清晰的可信边界划分，也为后续迁移打下了基础。

5.4.2 完整性校验与模型保护

系统在启动时会使用 SHA-256 对 SecureCore 的核心代码和模型文件进行完整性校验。系统会分别生成 `core_hash` 和 `model_hash`。这两个哈希值主要用于检测静态文件是否被篡改。测试结果表明，这两类哈希值都能够稳定生成，而且会随着文件内容变化而发生变化。这说明，该机制已经可以满足原型阶段对“核心组件没有被误改或恶意篡改”的基本核验需求。

还需要进一步说明的是，这一机制属于应用层的原型级完整性校验。它的主要作用，是为当前本地部署环境提供基本的防篡改能力，同时也为后续事件摘要附带可追溯的版本凭证。由于当前系统还没有接入真实硬件 TEE 的远程证明服务，所以这一机制并不等同于基于硬件根信任的标准 `attestation`。基于这一点，本作品在报告中将它界定为“远程证明迁移预留”的前置设计，而不是已经完成的硬件级可信证明链路。

表5-7 完整性校验测试结果

校验项	哈希算法	哈希长度	校验结果
<code>core_hash</code>	SHA-256	64 字符	d06b0be0...b8ffcb

model_hash	SHA-256	64 字符	0d718136...d53b4e
------------	---------	-------	-------------------

如表 5-7 所示，core_hash 和 model_hash 均正确生成 64 字符 SHA-256 摘要值。系统启动时自动执行完整性校验，篡改会导致哈希不匹配从而阻止运行。上述结果表明，当前原型已经实现了基于哈希的核心文件完整性校验机制。该机制可以用于本地部署阶段的原型级可信启动检查。这个结果主要用来说明 SecureCore 具备基本的防篡改设计能力，但它不能作为硬件 TEE 远程证明能力的替代证明。

5.4.3 事件摘要最小暴露

本测试主要用于验证 SecureCore 输出到外部业务层的事件摘要，是否真正遵循了最小暴露原则。如表 5-8 所示，测试结果表明，系统对外只输出 session_id、event_type、risk_level、anon_person_id、start_ts/end_ts、frame_count、core_hash 和 model_version 等必要字段。系统不会输出原始 tracker_id、完整轨迹序列，也不会输出未经脱敏的图像内容。这样的设计可以在满足告警处置、事件复核和统计分析需求的同时，尽量降低接口输出带来的隐私泄露风险。

测试结果还表明，摘要中附带的 core_hash 和 model_version 可以作为原型阶段的来源标识，用来支持事件生成过程的版本追溯。需要说明的是，这种“摘要级可追溯性”属于应用层的可验证设计。它并不等同于真实硬件 TEE 的远程证明令牌。它的主要作用，是增强当前原型在业务流程中的可核验性，也为后续迁移提供更好的兼容基础。

表5-8 事件摘要字段验证

字段	包含	说明
session_id	是	会话标识
event_type	是	事件类型
risk_level	是	风险等级
anon_person_id	是	匿名化人员 ID
start_ts/end_ts	是	起止时间
frame_count	是	持续帧数

core_hash	是	完整性校验值
model_version	是	模型版本
原始 tracker_id	否	已脱敏
完整 trajectory	否	已脱敏

5.5 差分隐私统计发布测试

5.5.1 统计接口 DP 接入测试

差分隐私模块采用 Laplace 机制对统计数据加噪，通过预算管理器控制隐私消耗，通过低频隐藏策略防止小样本推断攻击。隐私参数 $\epsilon=0.8$ 参考了行业实践中的常用配置，在隐私强度与数据可用性之间取得折中。

Laplace 加噪测试($\epsilon=0.8$, sensitivity=1.0)如表 5-9 所示，对真实值 100 加噪 10 次，均值约 100.7，噪声分布相对集中；对真实值 0 加噪 10 次均保持非负，验证了基础噪声特性。

表5-9 Laplace加噪测试结果

指标	值
真实值	100
加噪均值	100.7
标准差	1.0
加噪示例	101,100,99,101,100,100,103,101,101,101
非负约束(true=0)	全部非负

低频隐藏：计数值低于 5 的分类显示为"小于 5"。隐私预算管理测试结果如表 5-10 所示，预算管理(daily_limit=3.0, $\epsilon=0.8$)：第 4 次查询被拒绝，预算耗尽。

表5-10 隐私预算管理测试结果

查询次数	累计消耗	can_spend	结果
------	------	-----------	----

第 1 次	0.8	True	允许
第 2 次	1.6	True	允许
第 3 次	2.4	True	允许
第 4 次	3.2 超过 3.0	False	拒绝

StatsService 完整流程：原始 total=50 加噪后为 56，来源标记 fresh_dp。字典加噪 {FALLEN:10,STILLNESS:30,NIGHT_ABNORMAL:10} 加噪后求和从 50 变为 51。

5.5.2 DP 对系统可用性影响

DP 对可用性影响：（1）加噪偏差约 1-2%，可忽略；（2）每日最多 3 次查询，缓存 TTL=10min 减少消耗；（3）低频隐藏损失精确数字但防止推断攻击。整体影响可接受。

5.6 系统实时性与稳定性测试

系统实时性测试使用 480x640 随机帧输入，经预处理缩放至 320x320 后送入模型，连续处理 10 帧，测试结果如表 5-11 所示。

表5-11 单帧推理延迟测试结果

指标	值
测试帧数	10
平均延迟	11.3 ms
最小延迟	9.5 ms
最大延迟	13.8 ms
标准差	1.1 ms

单帧平均计算延迟稳定在 11.3 ms，对应理论帧率超 80fps。初步验证了该方案在本地 CPU 算力下具备较低延迟的实时处理能力。WebSocket 端到端延迟 66.3ms，100 帧测试无丢帧。

5.7 测试结论

(1) 模型在训练集、验证集和测试集上都取得了较好的检测效果。训练集的 mAP50 为 0.959, 验证集的 mAP50 为 0.934, 测试集的 mAP50 为 0.827。这说明模型已经具备一定的基础部署价值。

(2) 规则引擎中的帧数确认、时间升级和恢复检测机制都能够按预期工作。系统可以支持跌倒、静止和夜间异常等事件的多级风险判定。

(3) SecureCore 原型在普通 CPU 环境下完成了敏感链路封装、应用层完整性校验和事件摘要最小暴露验证。这个结果说明, 系统已经具备了面向 TEE 迁移的逻辑隔离基础。

(4) 差分隐私统计发布模块的加噪偏差较小。预算控制和低频隐藏策略也能够正常工作。因此, 该模块可以用于聚合统计结果的受控发布。

(5) 系统的单帧平均延迟为 11.3 ms, WebSocket 端到端延迟为 66.3 ms。这个结果说明, 系统可以满足本地实时监测场景下的响应要求。

第6章 作品总结

6.1 特色与创新点

隐私优先架构。系统全程本地处理视频数据, 采用 MediaPipe 人脸检测配合 GaussianBlur 模糊处理实现人脸匿名化, 遵循角色分级隔离, 对公共平台仅传输事件摘要而非原始图像, 对授权看护者侧开放经人脸模糊处理的实时验证视图, 有效保护用户隐私。

规则与机器学习融合。YOLO26 深度学习检测与规则引擎风险分级相结合, 既利用了深度学习在人体姿态检测上的优势, 又通过规则引擎实现了可解释的风险分级逻辑, 兼顾准确性与可解释性。

四级风险分级体系。系统定义 NORMAL、LOW、MEDIUM、HIGH 四级风险等级, 针对不同风险匹配差异化响应策略, 从本地提醒到紧急通知, 实现精细化风险管理。

用户可配置参数。检测参数开放配置，包括跌倒确认帧数、静止阈值、夜间时段等，适应不同用户的个性化需求和使用场景。

完整系统化实现。前端多页面覆盖监控、分析、管理与可视化，后端多模块协同，检测核心完整实现，从数据采集到告警响应形成全链路闭环，具备实际部署能力。

基于防逆向穷举的联邦式隐私保护机制。在平台端的差分隐私（DP）统计发布中，系统不仅引入了拉普拉斯层，更是独立研发了底层的“单日计算配额池（Budget Manager）”与“短时快照缓存效期”机制。该设计彻底阻断了试图通过高频无差别拉取数据接口、采用求平均值来逆向推导院内老人真实行为的攻击手段，构筑了兼具业务可用性与防探测攻击的坚固隐私屏障。

6.2 推广应用场景

家庭独居老人监护是本系统的核心应用场景，通过非接触式摄像头监测老人居家安全，及时发现跌倒、长时间静止等异常情况。养老院集中管理场景中，系统可部署于多个房间，实现统一监控与告警管理。社区居家养老服务场景中，系统可作为社区养老服务的支撑工具，为社区老人提供安全保障。

6.3 未来发展展望

检测能力拓展方面，可增加徘徊检测、异常区域滞留检测等更多异常行为识别能力。模型轻量化方面，可探索更轻量的检测模型降低硬件门槛，支持边缘设备部署。权限管理方面，可实现角色权限区分，支持管理员、护工、家属等多角色访问。告警渠道方面，可对接短信、电话、微信等告警接口，实现多渠道告警触达。

参考文献

- [1] 孔钦，吴昊阳. 基于轻量化 YOLOv8 的行人跌倒检测算法的研究与实现[J]. 现代信息技术, 2025.
- [2] 张玥. 老年人跌倒风险的六国对比研究及中国队列研究——基于 SAGE 调查数据的分析

- [D]. 广州：暨南大学，2023.
- [3] 成鑫才. 基于 SSD-M1 和 DeeperCut-S 的人体姿态识别研究[D]. 成都：西南交通大学，2023.
- [4] 彭芙蓉，卢张宇，赵一帆. 基于改进 ByteTrack 的行人跟踪算法[J]. 湖南工程学院学报(自然科学版)，2025.
- [5] 吴欣泽，李长文. 基于 OpenCV+MediaPipe 的人体姿态估计融合监测系统设计[J]. 电子技术与软件工程，2025.
- [6] 邹一唱，王雅婷，赵贞莲，徐宇庆，徐嘉睿. 基于 YOLOv10 和 OpenPose 的老人跌倒检测综述[J]. 数字技术与应用，2026.
- [7] 吴宗胜，黄奕，王芝怡，王茁轩，申越. 基于 YOLOv11 跌倒检测系统的设计与实现[J]. 咸阳师范学院学报，2025.
- [8] 郑韩京. Vue.js 前端开发基础与项目实战[M]. 北京：人民邮电出版社，2020.
- [9] Dumitre I A, Rugina S G, Lengyel L C, et al. Kubernetes Configuration Files Created with a Python Quart Web Interface for Real Life Scenarios of Kubernetes Deployments[J]. Acta Technica Napocensis, 2024.
- [10] 张庆根. 基于 WebSocket 的热电厂调度通信数据实时传输与异常检测策略[J]. 电气应用，2026.
- [11] 王彬. 基于 WebSocket 协议的网页即时通信系统研究与实现[J]. 无线互联科技，2023.
- [12] 崔泽男，张俊林，王泽强. 基于人体姿态估计的跌倒行为检测方法研究[J]. 工业控制计算机，2026.